

PostgreSQL Autovacuum Tuning: Expert Reference for Versions 14–18

TL;DR

- Across PG14→18, autovacuum evolved from "make it survive wraparound" (PG14 failsafe) to "make it cheap and predictable": PG16 added opportunistic page freezing plus `(vacuum_buffer_usage_limit)`, PG17 replaced the 1GB dead-TID array with a memory-efficient TID store (adaptive radix tree), and PG18 added eager freezing, a dead-tuple hard cap (`(autovacuum_vacuum_max_threshold)`), dynamically reloadable `(autovacuum_max_workers)`, and AIO-accelerated heap scans.
- The single most impactful operational fact: **before PG17, VACUUM's dead-TID storage was a single allocation limited to 1GB (≈179 million dead tuples in one pass, since a TID is 6 bytes), forcing multi-pass index scans; PG17+ removes that cap** — so pre-17 you tune per-table scale factors to keep dead tuples down, on 17+ you leave `(autovacuum_work_mem)` at -1 and multi-pass becomes rare (Sawada's PGConf benchmark showed a 100M-row/25M-dead-tuple table use 143MB on PG16 vs. 28MB on PG17, ~20x reduction).
- Most autovacuum "not working" incidents are not tuning problems but **held-back xmin horizon** (long transactions, abandoned replication slots, `(hot_standby_feedback)`, orphaned prepared xacts) — verify these first, before touching any GUC.

Key Findings

Per-version headline changes

- PG14:** Wraparound **failsafe** (`(vacuum_failsafe_age)`=1.6B, `(vacuum_multixact_failsafe_age)`=1.6B) that drops cost delay and skips index/heap vacuuming to race `relfrozenxid` forward; autovacuum can dynamically **skip index vacuuming** when few dead tuples; per-index detail in autovacuum logs; I/O timing in auto-vacuum/analyze logs. Regression caution: **14.0–14.3 CREATE/REINDEX CONCURRENTLY silent index corruption**, fixed in 14.4 (released June 16, 2022; the PG14.4 notes credit Álvaro Herrera: "An optimization added in v14 caused CREATE INDEX ... CONCURRENTLY and REINDEX ... CONCURRENTLY to sometimes miss indexing rows that were updated during the index build. Revert that optimization.").
- PG15:** Mostly infrastructural for vacuum — **shared-memory cumulative statistics** (stats collector process removed), `(log_autovacuum_min_duration)` **default changed from -1 to 10min** (release notes, verbatim: "This changes the default of `log_checkpoints` to on and that of `log_autovacuum_min_duration` to 10 minutes. This will cause even an idle server to generate some log output..."), extra detail in VACUUM VERBOSE/autovacuum logs, and more aggressive `relfrozenxid`/`relminmxid` advancement.
- PG16:** **Opportunistic page freezing during non-freeze vacuums** (PG16.0 release notes, verbatim: "During non-freeze operations, perform page freezing where appropriate (Peter Geoghegan) ... This makes full-table freeze vacuums less necessary."); `(vacuum_buffer_usage_limit)` GUC + `(BUFFER_USAGE_LIMIT)` option ("Allow control of the shared buffer usage by vacuum and analyze (Melanie Plageman) ... The default value is set by server variable `vacuum_buffer_usage_limit`, which also controls autovacuum." — **default 256kB in PG16**); autovacuum honors cost-delay changes **per-block instead of per-relation**; `(vacuum_defer_cleanup_age)` **removed**; HOT updates allowed when only BRIN-indexed columns change; `(pg_stat_io)`.
- PG17:** **TID store / adaptive radix tree** replaces the flat dead-TID array — removes the 1GB cap, cuts memory use ~20x, makes multi-pass index vacuuming rare; more compact WAL from vacuum; streaming I/O speeds heap scans and ANALYZE; `(pg_stat_progress_vacuum)` gains `(indexes_total)`/`(indexes_processed)`; `(MAINTAIN)` privilege / `(pg_maintain)` role; `(vacuum_buffer_usage_limit)` default raised to 2MB.
- PG18:** **AIO subsystem** (`(io_method)` worker/`io_uring`/sync) accelerates vacuum/analyze heap and bitmap scans; **eager freezing** in normal vacuum (`(vacuum_max_eager_freeze_failure_rate)`, default 0.03); `(autovacuum_vacuum_max_threshold)` hard dead-tuple cap (default 100,000,000); `(autovacuum_max_workers)` **now reloadable** (SIGHUP) with new `(autovacuum_worker_slots)` (restart, default 16) as the ceiling; per-table `(total_vacuum_time)`/`(total_autovacuum_time)`; `(track_cost_delay_timing)`; `(vacuum_truncate)` GUC; `(effective_io_concurrency)`/`(maintenance_io_concurrency)` defaults raised to 16.

The xmin-horizon problem (all versions)

VACUUM can only remove tuples whose deleting XID precedes the global xmin horizon. Four things hold it back: (1) long-running transactions on the primary; (2) abandoned/inactive replication slots (`(pg_replication_slots.xmin)`/`(catalog_xmin)`); (3) `(hot_standby_feedback)` standbys running long queries (`(pg_stat_replication.backend_xmin)`); (4) orphaned prepared transactions (`(pg_prepared_xacts)`). Symptom in logs: "N dead but not yet removable" with an old "removable cutoff". No amount of autovacuum tuning fixes this.

Details

1. Autovacuum behavior and changes per version

PostgreSQL 14. The marquee change is the **wraparound failsafe**. When a table's `(relfrozenxid)`/`(relminmxid)` age crosses

`vacuum_failsafe_age`/`vacuum_multixact_failsafe_age` (default 1.6 billion), VACUUM stops applying cost delay, bypasses further index and heap vacuuming, and disables the buffer access strategy so it can use all of shared buffers — racing to advance `relfrozenxid`. This is checked dynamically at intervals during a running VACUUM and typically fires inside an already-running anti-wraparound autovacuum. PG14 also made VACUUM able to **dynamically skip index vacuuming** when there are very few dead tuples (the mechanism reused by `INDEX_CLEANUP off` and by failsafe), added **per-index information to autovacuum logging**, and reports I/O timings for auto-vacuum/analyze when `track_io_timing` is on. `autovacuum_vacuum_insert_threshold` (default 1000) and `autovacuum_vacuum_insert_scale_factor` (default 0.2), introduced in PG13, are the insert-driven triggers relevant here for append-mostly tables. It also lowered `vacuum_cost_page_miss` from 10 to 2, making default vacuum less throttled. **Regression:** PostgreSQL 14.0–14.3 had a bug where `CREATE INDEX CONCURRENTLY`/`REINDEX CONCURRENTLY` could build corrupt indexes with missing entries; fixed in **14.4** (June 16, 2022). Detect with `pg_amcheck --heapallindexed`. [\(Azure Docs\)](#) [\(PostgreSQL\)](#)

PostgreSQL 15. No large behavioral change to the autovacuum trigger logic, but two infrastructural items matter. The **cumulative statistics system moved into shared memory** (the separate stats collector process and `stats_temp_directory` are gone); autovacuum reads these stats to decide what to process. `log_autovacuum_min_duration`'s **default changed from -1 (disabled) to 10 minutes** (600000ms) in the same commit that flipped `log_checkpoints` to `on` — worth resetting on resource-constrained servers. VACUUM VERBOSE and autovacuum log messages gained additional detail (Peter Geoghegan), and vacuum became more aggressive at advancing the oldest frozen/multixact IDs. `vacuum_defer_cleanup_age` still exists in PG15 (removed only in PG16). [\(PostgreSQL + 2\)](#)

PostgreSQL 16. The headline is **opportunistic page freezing**: during ordinary (non-freeze) vacuums PostgreSQL now freezes pages "where appropriate," which reduces how often expensive full-table anti-wraparound freeze scans are needed (Peter Geoghegan). New `vacuum_buffer_usage_limit` GUC and the `BUFFER_USAGE_LIMIT` option to VACUUM/ANALYZE/vacuumdb control the ring-buffer (Buffer Access Strategy) size — **default 256kB in PG16** (raised to 2MB in PG17). Autovacuum now **re-reads cost-delay settings at the start of each block** rather than only at the start of each relation (Melanie Plageman), so `pg_reload_conf()` changes take effect mid-table. `vacuum_defer_cleanup_age` was **removed** (Andres Freund) — obsolete since replication slots/`hot_standby_feedback`. HOT updates are now allowed when only BRIN-indexed columns change (Matthias van de Meent, Josef Simanek, Tomas Vondra). `pg_stat_io` gives per-context (including `vacuum`) I/O visibility. New VACUUM options `PROCESS_MAIN`, `SKIP_DATABASE_STATS`, `ONLY_DATABASE_STATS`. Note: PG16 did **not** add "parallel autovacuum workers," "anti-wraparound starvation" scheduling, or "wraparound-risk prioritization" — those are commonly misattributed (parallel VACUUM was PG13; prioritized/eager scheduling is PG18). [\(PostgreSQL\)](#) [\(PostgreSQL\)](#)

PostgreSQL 17. The **new memory management for VACUUM** is the defining change: the flat, `maintenance_work_mem`-bounded dead-TID array (bsearch-based lookup, a single allocation limited to 1GB ≈ 179M dead tuples regardless of how much memory you gave it) is replaced by a **TID store built on an adaptive radix tree** (Masahiko Sawada, John Naylor). Consequences: the 1GB cap is gone; the same workload uses a fraction of the memory (benchmarked ~20x less — 143MB→28MB on a 100M-row/25M-dead-tuple table); and multi-pass index vacuuming (triggered when the dead-TID set overflows memory and forces a full index scan per pass) becomes rare. WAL emitted by vacuum is also more compact, vacuuming of index-less relations is optimized, `vacuum_buffer_usage_limit` default rose to 2MB, streaming I/O speeds sequential heap scans and ANALYZE, `pg_stat_progress_vacuum` gained `indexes_total`/`indexes_processed`, and the `MAINTAIN` privilege / `pg_maintain` role allow delegating VACUUM/ANALYZE without ownership. Note `snapshot_too_old`/`old_snapshot_threshold` was removed in PG17. [\(PostgreSQL\)](#)

PostgreSQL 18. Four autovacuum-relevant additions. (1) **Asynchronous I/O (AIO)**: new `io_method` (worker default, `io_uring` on Linux, `sync` fallback) lets backends — including autovacuum workers, which share the VACUUM/ANALYZE code paths — queue multiple reads and overlap them with processing; benefits heap sequential scans, bitmap heap scans, and vacuum/analyze (read path only; writes and index scans not yet). (2) **Eager freezing/scanning**: a normal vacuum may proactively scan some all-visible-but-not-all-frozen pages to freeze them, amortizing the cost of future aggressive vacuums; tuned by `vacuum_max_eager_freeze_failure_rate` (docs verbatim: "The default is 0.03 (3%)... Successful page freezes are capped internally at 20% of the all-visible but not all-frozen pages in the relation."). (3) `autovacuum_vacuum_max_threshold` (docs verbatim: "a limit on the value calculated with `autovacuum_vacuum_threshold` and `autovacuum_vacuum_scale_factor`. The default is 100,000,000 tuples. If -1 is specified, autovacuum will not enforce a maximum.") puts a hard ceiling on the scale-factor formula so very large tables get vacuumed before accumulating enormous dead-tuple backlogs: `vacuum_threshold` = `Minimum(max_threshold, base_threshold + scale_factor * reltuples)`. (4) **Dynamic workers**: `autovacuum_max_workers` is now `SIGHUP`-reloadable, with a new `autovacuum_worker_slots` (default 16, restart-only) reserving the slot pool that caps it. Also: per-table `total_vacuum_time`/`total_autovacuum_time`/`total_analyze_time` in `pg_stat_all_tables`, `track_cost_delay_timing` for measuring actual throttle sleeps, the `vacuum_truncate` GUC (to control the ACCESS EXCLUSIVE tail-truncation that can stall hot standbys), VACUUM/ANALYZE now recurse into inheritance children by default (use `ONLY`), and `effective_io_concurrency`/`maintenance_io_concurrency` defaults raised from 1 to 16.

2. Known problems / pain points per version

- **Autovacuum falling behind on high-write tables:** default `autovacuum_vacuum_scale_factor=0.2` means a table must accumulate 20% dead tuples before vacuum triggers — on a large hot table that's a huge amount of bloat. Fix with per-table lower scale factors (or, in PG18, `autovacuum_vacuum_max_threshold`).

- **xmin horizon held back** (all versions): the #1 "vacuum runs but dead tuples remain" cause. Check `(pg_stat_activity)` (long xacts), `(pg_replication_slots)` (xmin/catalog xmin), `(pg_stat_replication.backend_xmin)` (hot_standby_feedback), `(pg_prepared_xacts)`.
- **Wraparound / anti-wraparound (aggressive) autovacuum**: at `(autovacuum_freeze_max_age)` (default 200M) an anti-wraparound autovacuum is forced even if autovacuum is off; it is **not auto-cancelled by conflicting locks** (unlike normal autovacuum). Warnings begin ~40M XIDs from wraparound; the database refuses writes with ~3M left.
- **Autovacuum cancelled by DDL**: normal autovacuum holds SHARE UPDATE EXCLUSIVE and yields to conflicting lock requests; a job that repeatedly runs conflicting DDL/ANALYZE can prevent a table from ever being vacuumed.
- **The pre-PG17 1GB dead-TID cap**: on a table that accrues >179M dead tuples between vacuums, vacuum must scan every index multiple times (once per memory-fill). Pre-17 the real fix is per-table scale factor, not memory. On 17+ this is largely gone.
- **PG14.0-14.3 REINDEX/CREATE INDEX CONCURRENTLY corruption** (fixed 14.4). Minor: a PG17.x point release fixed a theoretical use-after-free in parallel vacuum.

3. All settings that influence autovacuum

Autovacuum daemon & triggers (all `(sighup)` context unless noted; "per-table" = overridable via storage parameter):

- `(autovacuum)` — master switch, default `(on)`, sighup. Wraparound autovacuaums still run even if off.
- `(autovacuum_worker_slots)` — **PG18 new**, default 16, **postmaster/restart**; reserves the worker slot pool. `(PostgreSQL)`
- `(autovacuum_max_workers)` — default 3; **postmaster/restart through PG17, SIGHUP-reloadable in PG18** (capped by `(autovacuum_worker_slots)`).
- `(autovacuum_naptime)` — default 1min, sighup; launcher wakes every naptime/N databases.
- `(autovacuum_vacuum_threshold)` — default 50, sighup, per-table.
- `(autovacuum_vacuum_scale_factor)` — default 0.2, sighup, per-table.
- `(autovacuum_vacuum_insert_threshold)` — default 1000 (PG13+), sighup, per-table; -1 disables.
- `(autovacuum_vacuum_insert_scale_factor)` — default 0.2, sighup, per-table.
- `(autovacuum_vacuum_max_threshold)` — **PG18 new**, default 100,000,000, sighup, per-table; -1 disables.
- `(autovacuum_analyze_threshold)` — default 50, sighup, per-table.
- `(autovacuum_analyze_scale_factor)` — default 0.1, sighup, per-table.
- `(autovacuum_vacuum_cost_delay)` — default 2ms (was 20ms before PG12), sighup, per-table; -1 inherits `(vacuum_cost_delay)`.
- `(autovacuum_vacuum_cost_limit)` — default -1 (inherits `(vacuum_cost_limit)=200`), sighup, per-table; distributed proportionally across running workers.
- `(autovacuum_freeze_max_age)` — default 200,000,000, **postmaster/restart** (reducible per-table); range up to ~2B. `(PostgreSQL)`
- `(autovacuum_multixact_freeze_max_age)` — default 400,000,000, **postmaster/restart** (reducible per-table).
- `(autovacuum_work_mem)` — default -1 (inherits `(maintenance_work_mem)`), sighup; per-worker dead-TID memory. New workers pick up the value on start (so a change needs new workers, or terminating current ones, to take effect).
- `(maintenance_work_mem)` — default 64MB, sighup (also session-settable by superuser/user for manual VACUUM/CREATE INDEX/REINDEX).

Cost-based delay (manual VACUUM; sighup + session/per-table):

- `(vacuum_cost_delay)` — default 0 (manual vacuum throttling off), sighup/user. `(EnterpriseDB)`
- `(vacuum_cost_limit)` — default 200, sighup/user.
- `(vacuum_cost_page_hit)` — default 1. `(OneUptime)`
- `(vacuum_cost_page_miss)` — default 2 (was 10 before PG14). `(OneUptime)`
- `(vacuum_cost_page_dirty)` — default 20. `(OneUptime)`

Freeze cutoffs (sighup + session/per-table):

- `(vacuum_freeze_min_age)` — default 50,000,000 (effective capped at half `(autovacuum_freeze_max_age)`).
- `(vacuum_freeze_table_age)` — default 150,000,000 (capped at 95% of `(autovacuum_freeze_max_age)`); triggers an aggressive vacuum.
- `(vacuum_multixact_freeze_min_age)` — default 5,000,000.
- `(vacuum_multixact_freeze_table_age)` — default 150,000,000.

Failsafe & eager freeze:

- `(vacuum_failsafe_age)` — default 1.6B (PG14+), sighup.

- `vacuum_multixact_failsafe_age` — default 1.6B (PG14+), sighup.
- `vacuum_max_eager_freeze_failure_rate` — **PG18 new**, default 0.03, user context, per-table; 0 disables eager scanning.

Buffer & I/O:

- `vacuum_buffer_usage_limit` — **PG16 new**, default 256kB in PG16 / 2MB in PG17+, user context; 0=unlimited, else 128kB-16GB, capped to 1/8 shared_buffers.
- `vacuum_truncate` — **PG18 GUC** (storage param existed earlier), default true.
- `io_method` — **PG18**, default worker; **postmaster/restart**.
- `io_workers` — **PG18**, default 3, sighup (worker mode only).
- `effective_io_concurrency` — default 16 in PG18 (was 1), user context.
- `maintenance_io_concurrency` — default 16 in PG18, user context; governs vacuum/analyze prefetch concurrency. (Neon)

Logging/monitoring:

- `log_autovacuum_min_duration` — default 10min (PG15+; was -1 before), sighup, per-table. (PostgreSQL)
- `track_cost_delay_timing` — **PG18 new**, default off, sighup.

Per-table storage parameters: `autovacuum_enabled`, `autovacuum_vacuum_threshold`, `autovacuum_vacuum_scale_factor`, `autovacuum_vacuum_insert_threshold`, `autovacuum_vacuum_insert_scale_factor`, `autovacuum_vacuum_max_threshold` (PG18), `autovacuum_analyze_threshold`, `autovacuum_analyze_scale_factor`, `autovacuum_freeze_min_age`, `autovacuum_freeze_max_age`, `autovacuum_freeze_table_age`, `autovacuum_multixact_freeze_min_age`, `autovacuum_multixact_freeze_max_age`, `autovacuum_vacuum_cost_delay`, `autovacuum_vacuum_cost_limit`, `vacuum_truncate`, `vacuum_max_eager_freeze_failure_rate` (PG18), plus `toast.*` equivalents for the TOAST table. A worker processing a table whose per-table `autovacuum_vacuum_cost_delay`/`cost_limit` is set is **excluded from the multi-worker cost-balancing** algorithm (giving that table an independent I/O budget).

4. Best practices for tuning

- **High-write OLTP hot tables:** lower per-table scale factor (0.01–0.05), set a modest threshold, raise cost limit / lower cost delay, and consider `fillfactor<100` to enable HOT updates (avoids index churn and makes vacuum cheaper). On PG18 use `autovacuum_vacuum_max_threshold` per table as a safety cap.
- **Append-mostly/insert-heavy:** rely on `autovacuum_vacuum_insert_threshold`/`insert_scale_factor` to trigger vacuums that update the visibility map and freeze; on PG18 raise `vacuum_max_eager_freeze_failure_rate` on those tables so freezing is amortized and future aggressive vacuums are short.
- **Large data-warehouse tables:** they may never hit dead-tuple thresholds, so the anti-wraparound vacuum dominates — reduce per-table `autovacuum_freeze_max_age`/`vacuum_freeze_table_age` so freezing is spread out, or on PG18 lean on eager freezing.
- **Heavy update/delete churn:** raise workers + cost limit, lower scale factors; watch `pg_stat_progress_vacuum.index_vacuum_count>1` (pre-17 sign of memory-driven multi-pass — fix with scale factor or `autovacuum_work_mem`).
- **Making autovacuum more aggressive:** lower scale factors, raise `autovacuum_vacuum_cost_limit` (1000–2000 on modern storage), lower `autovacuum_vacuum_cost_delay` (1ms SSD / 0 NVMe), raise `autovacuum_max_workers` **but** raise cost limit proportionally since the budget is shared across all running workers. Prefer per-table overrides over global changes.
- **Wraparound prevention:** monitor `age(relfrozenxid)` per table and `age(datfrozenxid)` per database; tune freeze ages; let failsafe be the last line. Never raise `autovacuum_freeze_max_age` above ~1B.
- **Memory sizing:** pre-PG17, `maintenance_work_mem`/`autovacuum_work_mem` above ~1GB is wasted for vacuum's dead-TID storage; set 1GB on big heavily-updated tables and control dead tuples via scale factor. PG17+, leave `autovacuum_work_mem=-1`; the radix tree does the same work in far less memory.
- **Cost-based delay interaction:** with the inherited limit 200 and 2ms delay, a single worker accumulates ~100,000 cost units/sec — roughly 800MB/s of buffer-hit reads, 400MB/s of miss reads, or 40MB/s of dirtied pages (a weighted mix in practice). The 200 default is a museum piece calibrated for 2008 hardware; 1000–2000 is a sane modern baseline.
- **Monitoring:** `pg_stat_progress_vacuum` (phase, heap/index progress), `pg_stat_user_tables` (`n_dead_tup`, `n_live_tup`, `last_autovacuum`, `autovacuum_count`), `pg_stat_activity` for running autovacuum, `log_autovacuum_min_duration=0` during tuning, catalog `age()` queries, `pgstattuple` for real bloat, PG18 `pg_aio`/`pg_stat_io` for AIO and per-table `total_autovacuum_time`.
- **PG18 AIO/eager:** on cloud/network storage set `io_method=worker` (or `io_uring` on modern Linux) and raise `maintenance_io_concurrency`; use eager freezing on append-heavy tables; use `vacuum_truncate=off` on tables feeding busy hot standbys to avoid replay stalls.

5. Recommended starting-point settings (busy production OLTP)

Global (`postgresql.conf`), all sighthup unless marked):

```
autovacuum = on
autovacuum_max_workers = 5          # restart pre-18; sighthup in 18 (set worker_slots >= this)
autovacuum_worker_slots = 16        # PG18 only, restart
autovacuum_naptime = 15s            # lower if many databases
autovacuum_vacuum_scale_factor = 0.1 # from 0.2
autovacuum_analyze_scale_factor = 0.05
autovacuum_vacuum_insert_scale_factor = 0.1
autovacuum_vacuum_cost_limit = 2000  # from effective 200
autovacuum_vacuum_cost_delay = 2ms    # 1ms SSD / 0 NVMe
maintenance_work_mem = 1GB           # meaningful cap pre-17; on 17+ radix tree uses far less
autovacuum_work_mem = -1
log_autovacuum_min_duration = 0       # during tuning; raise later
# PG18:
autovacuum_vacuum_max_threshold = 100000000
vacuum_max_eager_freeze_failure_rate = 0.03
io_method = worker                   # restart
maintenance_io_concurrency = 32
```

Per-table for hot tables: (`autovacuum_vacuum_scale_factor=0.01`), (`autovacuum_vacuum_threshold=1000`), (`autovacuum_vacuum_cost_delay=0`), and set a per-table cost limit so the worker gets an independent budget (excluded from balancing). Reasoning: defaults were calibrated for 2008-era spinning disks; the shared cost budget means adding workers without raising the limit doesn't speed things up; per-table overrides let you be aggressive on hot tables without punishing the whole cluster. Restart-required items to plan for: (`autovacuum_max_workers`) (pre-18), (`autovacuum_worker_slots`) (18), (`autovacuum_freeze_max_age`), (`autovacuum_multixact_freeze_max_age`), (`io_method`).

6. Per-version cheat-sheet (quick reference)

PG14

- New: wraparound **failsafe** (`vacuum_failsafe_age`/`vacuum_multixact_failsafe_age`) = 1.6B; dynamic index-vacuum skipping; per-index autovacuum logging; auto-vacuum/analyze I/O timings; (`vacuum_cost_page_miss`) lowered 10→2.
- Defaults: freeze_max_age 200M, multixact 400M, cost_delay 2ms, cost_limit 200, scale_factor 0.2, log_autovacuum_min_duration -1.
- Gotchas: **14.0-14.3 CONCURRENTLY index corruption** → upgrade to 14.4, (`pg_amcheck`); 1GB dead-TID cap → multi-pass on huge tables; anti-wraparound autovacuum not lock-cancellable.

PG15

- New: shared-memory stats (no stats collector); (`log_autovacuum_min_duration`) default -1→10min (with (`log_checkpoints=on`)); richer VACUUM VERBOSE/autovacuum logs; more aggressive relfrozenxid/relminmxid advance.
- Defaults: as PG14 except log_autovacuum_min_duration now 10min. (`vacuum_defer_cleanup_age`) still present.
- Gotchas: idle servers now log autovacuum/checkpoints — reset on small hosts; still 1GB dead-TID cap.

PG16

- New: **opportunistic freezing in non-freeze vacuums**; (`vacuum_buffer_usage_limit`) (**256kB**) + (`BUFFER_USAGE_LIMIT`); cost-delay reload **per-block**; (`vacuum_defer_cleanup_age`) **removed**; BRIN-only HOT updates; (`pg_stat_io`); PROCESS_MAIN/SKIP_DATABASE_STATS/ONLY_DATABASE_STATS.
- Defaults: buffer_usage_limit 256kB; others as PG15.
- Gotchas: still 1GB dead-TID cap (last version with it); don't misattribute parallel-autovacuum/wraparound-scheduling here.

PG17

- New: **TID store (adaptive radix tree)** — 1GB cap gone, ~20x less memory, multi-pass rare; compact vacuum WAL; streaming I/O for heap scans/ANALYZE; index progress in (`pg_stat_progress_vacuum`); (`MAINTAIN`)/(`pg_maintain`); (`vacuum_buffer_usage_limit`) default **2MB**.
- Defaults: buffer_usage_limit 2MB; (`old_snapshot_threshold`) removed.
- Gotchas: (`autovacuum_work_mem`)/(`maintenance_work_mem`) no longer the vacuum bottleneck → leave (`autovacuum_work_mem=-1`); watch (`index_vacuum_count>1`) (now uncommon → investigate).

PG18

- New: **AIO** (`io_method`) worker/io_uring/sync) for vacuum/analyze reads; **eager freezing** (`vacuum_max_eager_freeze_failure_rate` 0.03, success cap 20%); `autovacuum_vacuum_max_threshold` (100M hard cap); `autovacuum_max_workers` **SIGHUP-reloadable** + `autovacuum_worker_slots` (16, restart); per-table `total_(auto)vacuum_time`; `track_cost_delay_timing`; `vacuum_truncate` GUC; VACUUM/ANALYZE recurse into children (use `ONLY`); `effective_io_concurrency`/`maintenance_io_concurrency` **1→16**.
- Defaults: eio/mio 16, io_method worker, io_workers 3.
- Gotchas: AIO read-path only (no writes/index scans yet); `io_uring` has kernel/security caveats; new-at-GA — verify against installed minor; tail-truncation ACCESS EXCLUSIVE can stall hot standbys → `vacuum_truncate=off`.

Recommendations

1. **Before tuning anything**, confirm the xmin horizon is advancing (four checks above). A stuck slot or long transaction masquerades as an autovacuum tuning problem.
2. Set `log_autovacuum_min_duration=0` temporarily and read the actual autovacuum log lines (dead-but-not-removable, removable cutoff, index scans count) to diagnose before changing GUCs.
3. **Raise the cost budget first** (`autovacuum_vacuum_cost_limit` to 1000-2000, delay to ≤1ms on SSD/NVMe) — the biggest single win on modern hardware. Then add workers with proportional budget.
4. **Use per-table overrides** for the handful of hot/large tables rather than blunt global changes.
5. **On PG17+**, stop treating `maintenance_work_mem`/`autovacuum_work_mem` as the vacuum bottleneck; leave `autovacuum_work_mem=-1`. On PG16 and earlier, cap at 1GB and manage dead tuples via scale factor.
6. **On PG18**, adopt `autovacuum_vacuum_max_threshold` and eager freezing for large tables, enable AIO on cloud storage, and use dynamic `autovacuum_max_workers` for bloat emergencies without a restart.

Thresholds that change the plan: if `pg_stat_progress_vacuum.index_vacuum_count>1` recurs → raise memory (pre-17) or lower scale factor; if `age(relfrozenxid)` climbs toward `autovacuum_freeze_max_age` → freezing is blocked, investigate xmin holders; if all workers are always busy (`count(*)` from `pg_stat_activity` where `query like 'autovacuum:%'` = `max_workers`) → add workers/budget.

Caveats

- Version-18 features (AIO, eager freezing, `autovacuum_vacuum_max_threshold`, dynamic `autovacuum_max_workers`) are new; several sources are from beta and details may have shifted at GA — verify against the installed minor version.
- `vacuum_buffer_usage_limit` default differs by version (256kB in PG16, 2MB in PG17+) — a common source of confusion.
- The "1GB / ~179M dead tuple" figures are the widely cited approximations of the old single-allocation array's 32-bit element-count limit (a TID is 6 bytes); exact counts depend on TID density.
- Cloud-managed Postgres (RDS/Aurora, Azure, Cloud SQL) may override defaults, add adaptive autovacuum tuning (e.g., RDS scaling `autovacuum_max_workers` on PG18), and restrict which parameters are settable.
- `autovacuum_worker_slots` "default 16" reflects the documented typical value; initdb may pick a lower number if kernel limits require it.